RAPORT
BREAK THE LOGIN

CUPRINS

1. Introducere 2
    1.1. Descrierea Aplicatiei 2
    1.2. Arhitectura Tehnica 3
2. Setup Mediu de Lucru 3
    2.1. Configurare Masina Virtuala 3
    2.2. Instalare si Configurare 4
    2.3. Instalare Instrumente de Securitate 5
3. Implementare MVP - v1 6
    3.1. Structura Bazei de Date 6
    3.2. Prezentarea Arhitecturii Codului 7
4. Prezentarea si Demonstrarea Vulnerabilitatilor 8
    4.1. Password Policy (OWASP: A7:2021) 8
    4.2. Stocarea Parolelor in Clar (OWASP: A2:2021) 8
    4.3. User Enumeration (OWASP: A7:2021) 10
    4.4. Brute-Force Attack (OWASP: A7:2021) 12
    4.5. Gestionarea Nesigura a Sesiunilor (OWASP: A7:2021) 15
    4.6. Resetarea Parolei Nesigura (OWASP: A7:2021) 16
5. Implementarea Masurilor de Securitate - v2 17
    5.1. Password Policy 17
    5.2. Prevenire User Enumeration 18
    5.3. Protectie Impotriva Atacurilor Brute-Force 20
    5.4. Gestionarea Securizata a Sesiunilor 22
    5.5. Mecanism Sigur de Resetarea Parolei 23
6. Audit si Logging 24
7. Concluzii 26

1. Introducere

1.1. Descrierea Aplicatiei

Acest raport documenteaza procesul de construire, testare de securitate si securizare a unei aplicatii web denumita "Break the Login". Aplicatia este un sistem simplu de management al tichetelor de suport, proiectat initial cu vulnerabilitati de securitate comune pentru a servi ca studiu de caz practic. Obiectivul proiectului este de a demonstra intelegerea mecanismelor de atac specifice aplicatiilor web si de a implementa solutii corecte si eficiente pentru remedierea acestora.

1.2. Arhitectura Tehnica

Limbaj de Programare	Python 3.11
Framework Web	Flask
Baza de Date	SQLite 3
OS	Debian 12 “Bookworm”
Platforma de Virtualizare	Oracle Virtual Box

2. Setup Mediu de Lucru

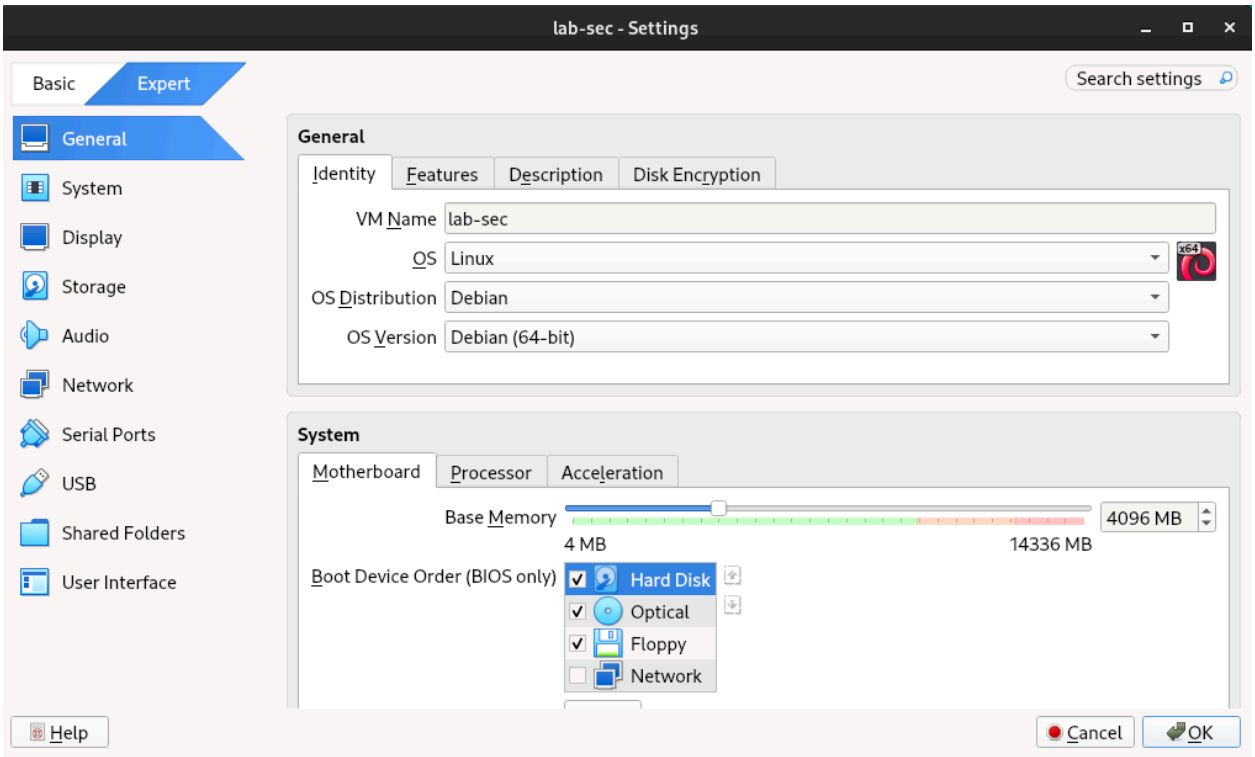
2.1. Configurare Masina Virtuala

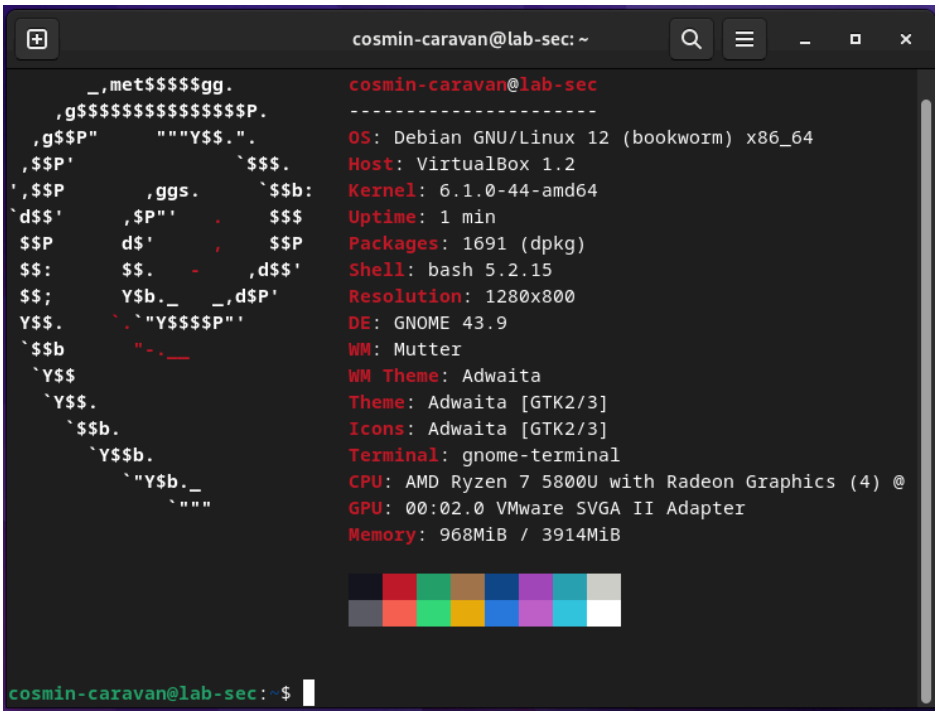
Pentru acest proiect, am configurat mediul de lucru pe o masina virtuala noua pentru a asigura un mediu izolat si reproductibil.

1. **Instalare Sistem de Operare:** Am instalat distributia Debian 12 “Bookworm”.

2. **Creare Utilizator:** Am creat utilizatorul cosmin-caravan si l-am adaugat in grupul sudoers.

3. **Actualizare Sistem:** Am intampinat o problema la actualizarea pachetelor, deoarece sursele apt pointau catre imaginea ISO. Am remediat problema prin editarea fisierului /etc/apt/sources.list si rularea sudo apt update && sudo apt upgrade -y.



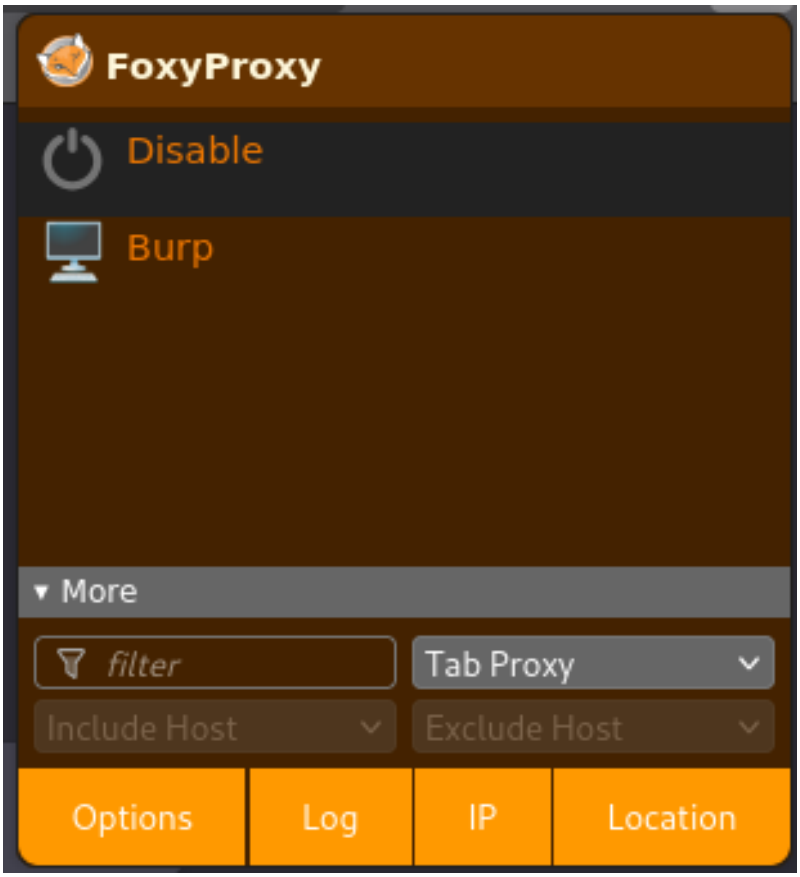


2.2. Instalare si Configurare

1. **Instalare Unelte Esentiale:** Am instalat pachetele necesare folosind comanda: sudo apt install -y python3 python3-pip python3-venv git sqlite3
2. **Creare Mediu Virtual si Instalare Biblioteci Python:** In directorul proiectului, am creat un mediu virtual si am instalat bibliotecile Python necesare:
   python3 -m venv venv
   source venv/bin/activate
   pip install flask flask-bcrypt flask-limiter itsdangerous

2.3. Instalare Instrumente de Securitate

1. **Burp Suite Community Edition:** Am descarcat si instalat Burp Suite pentru a intercepta si manipula traficul HTTP.
2. **FoxyProxy:** Am instalat extensia FoxyProxy in browser pentru a comuta usor traficul prin proxy-ul Burp.
3. **Certificat Burp:** Am instalat certificatul CA al Burp in browser pentru a putea intercepta traficul HTTPS.



3. Implementare MVP - v1

Am construit versiunea initiala a aplicatiei in mod intentionat cu functionalitati de baza si fara masuri de securitate adecvate.

3.1. Structura Bazei de Date

Pentru versiunea initiala a aplicatiei (MVP), am proiectat o structura de baze de date in SQLite, menita sa fie functionala, dar intentionat vulnerabila. Baza de date, denumita v1.db, contine trei tabele principale: users pentru managementul conturilor, tickets pentru gestionarea tichetelor de suport si audit_logs pentru inregistrarea evenimentelor.

Tabela **users**:

```
CREATE TABLE users (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   email TEXT UNIQUE NOT NULL,
   password TEXT NOT NULL,
   role TEXT CHECK(role IN ('ANALYST', 'MANAGER')) DEFAULT
'ANALYST',
   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
   locked BOOLEAN DEFAULT 0
);
```

In mod deliberat, am ales sa stochez parola in plain text (clar). Aceasta decizie introduce direct vulnerabilitatea A2:2021 – Cryptographic Failures, deoarece o compromitere a bazei de date ar expune instantaneu toate parolele.

Tabela **tickets**:

```
CREATE TABLE tickets (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   title TEXT NOT NULL,
   description TEXT,
   severity TEXT CHECK(severity IN ('LOW', 'MED', 'HIGH')) DEFAULT
'LOW',
```

```
    status TEXT CHECK(status IN ('OPEN', 'IN PROGRESS', 'RESOLVED'))
DEFAULT 'OPEN',
    owner_id INTEGER,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY(owner_id) REFERENCES users(id)
);
```

Tabela **audit_logs**:

```
CREATE TABLE audit_logs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER,
    action TEXT NOT NULL,
    resource TEXT,
    resource_id TEXT,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    ip_address TEXT,
    FOREIGN KEY(user_id) REFERENCES users(id)
);
```

Desi tabela audit_logs exista in structura v1, implementarea logarii in codul aplicatiei Python inexistenta. Evenimente critice de securitate, precum tentativele de login esuate, accesul neautorizat la resurse sau modificarile de parole, nu sunt inregistrate corespunzator. Aceasta lipsa de trasabilitate face dificila detectarea unui atac in desfasurare sau analiza post-incident, fiind o slabiciune in sine. Imbunatatirea si utilizarea efectiva a acestui mecanism vor fi abordate in versiunea v2.

3.2. Prezentarea Arhitecturii Codului

Arhitectura aleasa pentru v1 este deliberat simplista si nesigura. Prin concentrarea logicii intr-un singur fisier si prin ignorarea bunelor practici de securitate (cum ar fi hashing-ul parolelor, utilizarea de query-uri parametrizate, validarea accesului la resurse si managementul securizat al sesiunilor), am creat un mediu perfect pentru a demonstra si exploata vulnerabilitatile discutate in sectiunile urmatoare.

Toate operatiunile CRUD (Create, Read, Update, Delete) sunt realizate folosind biblioteca sqlite3. Query-urile SQL sunt adesea construite prin concatenare de string-uri, ceea ce, in alte contexte, ar deschide usa catre atacuri de tip **SQL Injection**.

4. Prezentarea si Demonstrarea Vulnerabilitatilor

4.1. Password Policy (OWASP: A7:2021)

Aplicatia, in versiunea sa initiala, nu impune nicio regula privind complexitatea parolelor la crearea unui cont nou. Utilizatorii au libertatea de a alege parole extrem de simple, scurte si usor de ghicit, precum "123", "password" sau "test".

Aceasta neglijenta slabeste fundamental securitatea conturilor, chiar si inainte de a discuta despre stocarea lor. Un atacator poate exploata aceasta slabiciune pentru a sparge conturi mult mai usor prin atacuri de tip Brute-Force sau Password Spraying.

Conturile cu parole slabe pot fi compromise in cateva secunde sau minute folosind un atac de fortare a parolei cu un dictionar de parole comune.

4.2. Stocarea Parolelor in Clar (OWASP: A2:2021)

Aplicatia v1 comite una dintre cele mai grave erori de securitate: stocarea parolelor utilizatorilor in format plain text (clar), direct in baza de date. In tabela users, coloana password contine exact secventa de caractere pe care utilizatorul a introdus-o la inregistrare, fara a se aplica niciun mecanism de hashing sau criptare.

Aceasta abordare incalca principiul fundamental de "defense in depth". Securitatea tuturor conturilor depinde exclusiv de securitatea bazei de date. Daca un atacator reuseste sa obtina acces la fisierul v1.db (printr-o alta vulnerabilitate, cum ar fi SQL Injection, Path Traversal, sau printr-un acces neautorizat la sistemul de fisiere), el obtine instantaneu acces la toate acreditarile utilizatorilor.

PoC: Pentru a demonstra aceasta vulnerabilitate, simulam un scenariu in care un atacator a obtinut acces la sistemul de fisiere al serverului unde ruleaza aplicatia.

```
cosmin-caravan@lab-sec:~/Downloads/auth-security-lab/v1$ sqlite3 v1.db
SQLite version 3.40.1 2022-12-28 14:03:47
Enter ".help" for usage hints.
sqlite> .schema
CREATE TABLE users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    email TEXT UNIQUE NOT NULL,
    password TEXT NOT NULL,
    role TEXT CHECK(role IN ('ANALYST', 'MANAGER')) DEFAULT 'ANALYST',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    locked BOOLEAN DEFAULT 0
);
CREATE TABLE sqlite_sequence(name,seq);
CREATE TABLE tickets (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    description TEXT,
    severity TEXT CHECK(severity IN ('LOW', 'MED', 'HIGH')) DEFAULT 'LOW',
    status TEXT CHECK(status IN ('OPEN', 'IN PROGRESS', 'RESOLVED')) DEFAULT 'OPEN',
    owner_id INTEGER,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY(owner_id) REFERENCES users(id)
);
CREATE TABLE audit_logs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER,
    action TEXT NOT NULL,
    resource TEXT, -- ticket, auth, audit
    resource_id TEXT,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    ip_address TEXT,
    FOREIGN KEY(user_id) REFERENCES users(id)
```

```
cosmin-caravan@lab-sec:~/Downloads/auth-security-lab/v1$ sqlite3 v1.db
SQLite version 3.40.1 2022-12-28 14:03:47
Enter ".help" for usage hints.
sqlite> select * from users;
1|admin@test.com|admin1234|MANAGER|2026-04-26 10:07:22|0
2|cosmin@test.com|test123|ANALYST|2026-04-26 10:07:22|0
sqlite>
```

4.3. User Enumeration (OWASP: A7:2021)

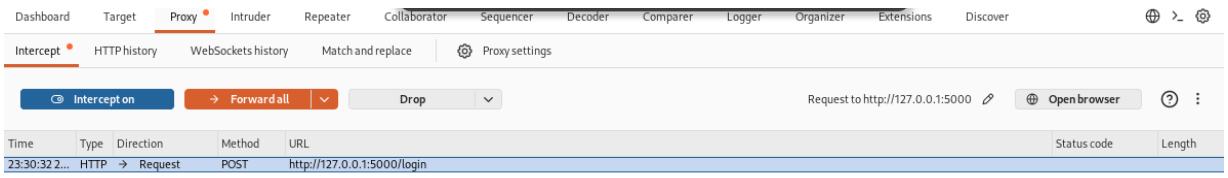
Vulnerabilitatea de User Enumeration apare atunci cand o aplicatie ofera raspunsuri diferite in functie de existenta sau inexistenta unui cont de utilizator in sistem. In cazul aplicatiei v1, formularul de login se comporta diferit in urmatoarele doua scenarii:

1. Cand un utilizator introduce un email care nu exista in baza de date.
2. Cand un utilizator introduce un email care exista, dar parola este gresita.

Aceasta diferenta in mesaje, oricat de mica, permite unui atacator sa "ghiceasca" sau sa enumere o lista de adrese de email valide, inregistrate in aplicatie.

```
(venv) cosmin-caravan@lab-sec:~/Downloads/auth-security-lab/v1$ python3 main.py
 * Serving Flask app 'main'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 108-155-359
```

Configuram browser-ul sa treaca prin proxy-ul Burp Suite si incercam sa ne logam cu un email si o parola oarecare. Interceptam cererea POST catre /login.

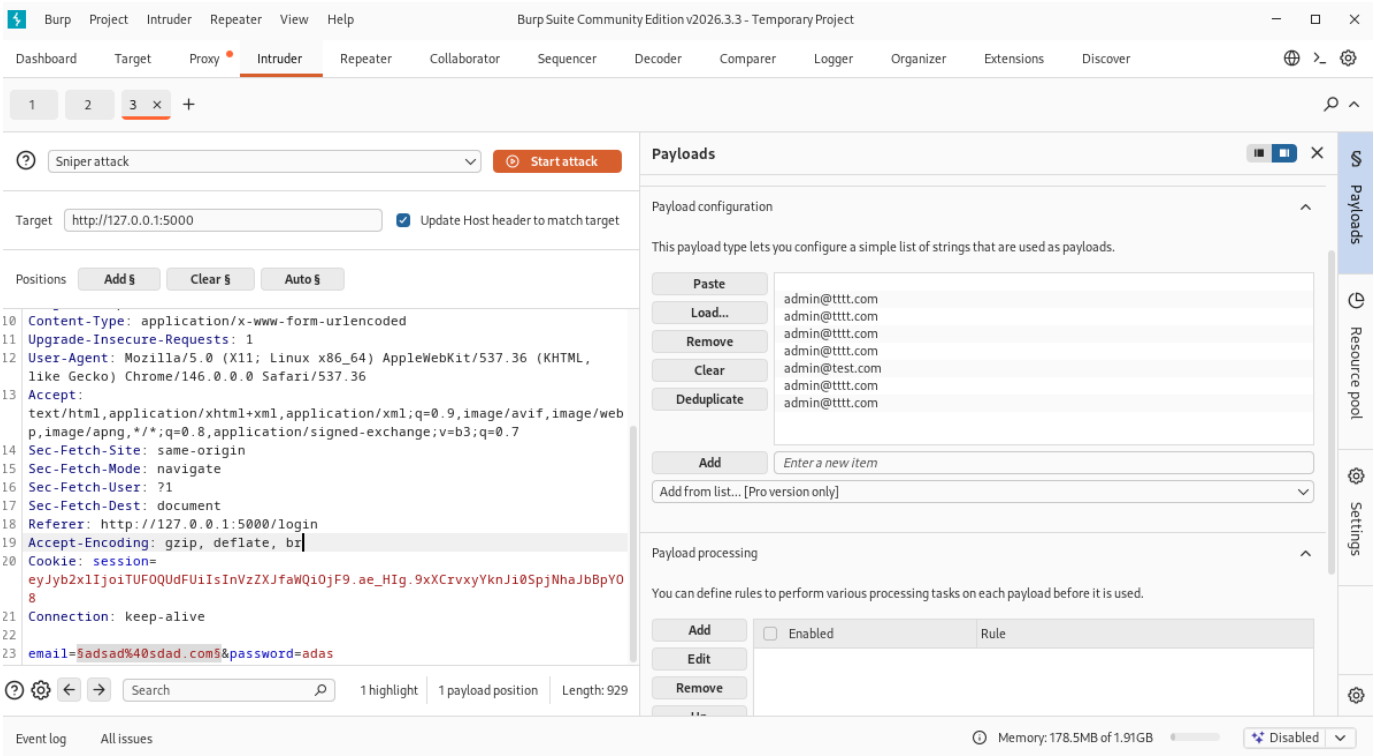


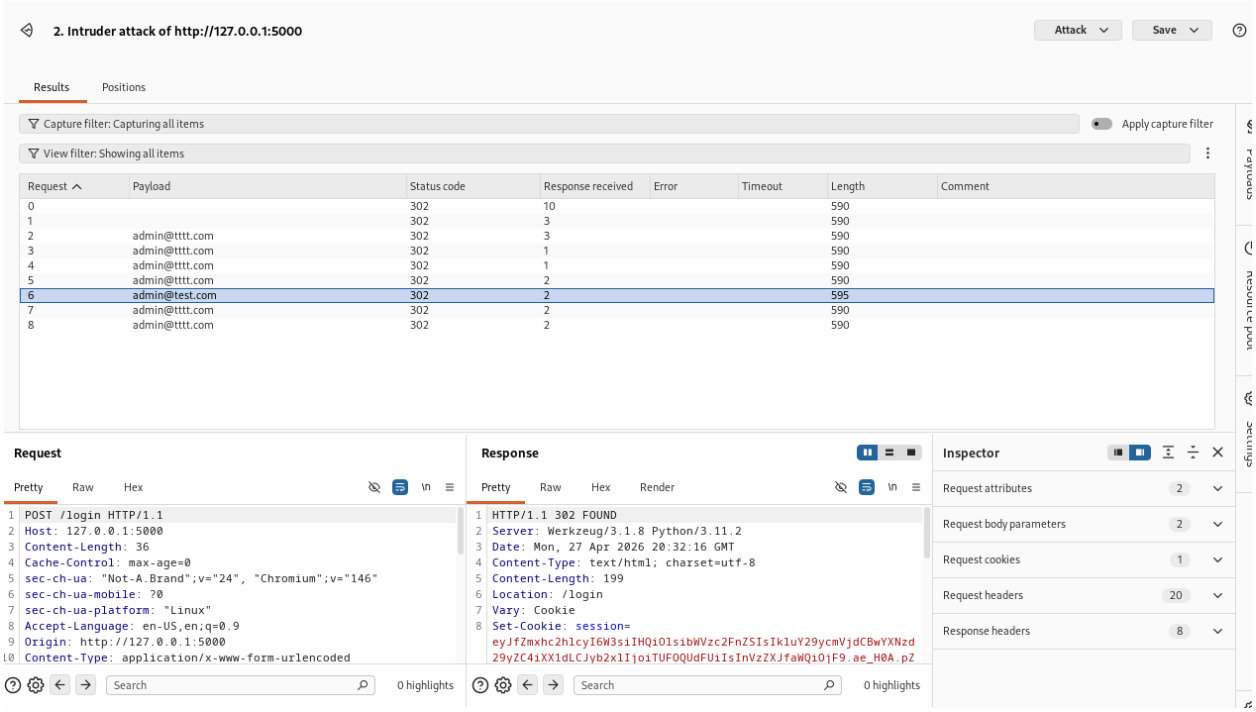
Trimitem cererea interceptata catre Burp Intruder (Click dreapta -> "Send to Intruder"). In tab-ul "Positions", ne asiguram ca singurul punct de injectie (§...§) este pe valoarea parametrului email.

In tab-ul "Payloads", incarcam o lista de potentiale adrese de email (un "wordlist"). Aceasta lista poate contine atat email-uri care stim ca exista in sistem, cat si email-uri invalide.

Lansam atacul. Burp Intruder va trimite o cerere de login pentru fiecare email din lista. Dupa finalizarea atacului, analizam rezultatele. Vom observa o diferenta clara in raspunsurile primite de la server.

Putem sorta rezultatele dupa coloana "Length" (lungimea raspunsului HTTP). Raspunsurile pentru email-urile valide vor avea o lungime diferita fata de cele pentru email-urile invalide, deoarece contin mesaje de eroare diferite.





4.4. Brute-Force Attack (OWASP: A7:2021)

Un atac de tip Brute-Force asupra unui formular de autentificare consta in incercarea sistematica a unui numar mare de parole pentru un singur nume de utilizator, in speranta ca una dintre ele va fi corecta.

Aplicatia v1 este extrem de vulnerabila la acest tip de atac din doua motive principale:

1. Lipsa Mecanismelor de Protectie: Formularul de login nu implementeaza nicio masura de aparare, cum ar fi:
2. Rate Limiting: Nu exista o limita a numarului de incercari de login esuate intr-un anumit interval de timp.
3. Account Lockout: Conturile nu sunt blocate temporar dupa un numar de incercari esuate.
4. CAPTCHA: Nu se solicita nicio verificare suplimentara pentru a demonstra ca utilizatorul este uman.

Existenta Vulnerabilitatii de User Enumeration: Dupa cum am demonstrat in sectiunea anterioara, un atacator poate mai intai sa identifice un email valid (o tinta), iar apoi sa isi concentreze atacul Brute-Force asupra acelui cont specific.

Combinatia acestor slabiciuni permite unui atacator sa ruleze un script automatizat care poate testa mii de parole pe minut, facand compromiterea unui cont cu o parola slaba sau comuna doar o chestiune de timp.

PoC:

Vom folosi din nou Burp Suite Intruder pentru a simula un atac Brute-Force asupra unui cont valid, pe care l-am descoperit anterior prin User Enumeration (de exemplu, admin@test.com).

Ca si in cazul atacului de User Enumeration, interceptam o cerere POST catre /login.

In tab-ul "Positions", setam email-ul la o valoare fixa (tinta noastra: admin@test.com). Adaugam un nou punct de injectie (§...§) doar pe valoarea parametrului password.

In tab-ul "Payloads", incarcam o lista de parole comune (un "password wordlist"). Aceasta lista poate contine sute sau mii de parole, inclusiv parola corecta a contului tinta (admin123).

Lansam atacul. Burp Intruder va trimite o cerere de login pentru fiecare parola din lista. Pentru a identifica parola corecta, analizam raspunsurile.

Vom observa ca majoritatea cererilor esuate vor returna un raspuns cu o anumita lungime ("Length"). Insa, cererea care foloseste parola corecta (admin123) va primi un raspuns diferit de la server (o redirectionare catre /dashboard), care va avea o lungime semnificativ diferita.

Chiar daca atacul nu reuseste, un numar foarte mare de cereri de login poate suprasolicita serverul si baza de date, ducand la o incetinire sau blocare a aplicatiei pentru utilizatorii legitimi.

```
(venv) cosmin-caravan@lab-sec:~/Downloads/auth-security-lab/v1$ python3 main.py
 * Serving Flask app 'main'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 108-155-359
```

Sniper attack | Start attack

Target: http://127.0.0.1:5000 | Update Host header to match target

Positions: Add § | Clear § | Auto §

```
POST /login HTTP/1.1
Host: 127.0.0.1:5000
Content-Length: 37
Cache-Control: max-age=0
sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Linux"
Accept-Language: en-US,en;q=0.9
Origin: http://127.0.0.1:5000
Content-Type: application/x-www-form-urlencoded
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif
,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b
3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer: http://127.0.0.1:5000/login
Accept-Encoding: gzip, deflate, br
Connection: keep-alive

email=admin%40test.com&password=§dasad§
```

Search | 1 highlight | 1 payload position | Length: 837

Payloads

Payload position: All payload positions

Payload type: Simple list

Payload count: 6

Request count: 6

Payload configuration

This payload type lets you configure a simple list of strings that are used as payloads.

Paste | Load... | Remove | Clear | Deduplicate | Add

asdad
sasa222
asdadkkakaka
admin1234
septe
admin1234

Add from list... [Pro version only]

Payload processing

You can define rules to perform various processing tasks on each payload before it is used.

Add | Edit | Remove

Enabled | Rule

Payloads | Resource pool | Settings

Results | Positions

Capture filter: Capturing all items | Apply capture filter

View filter: Showing all items

Request	Payload	Status code	Response received	Error	Timeout	Length	Comment
0		302	7			557	
1	asdad	302	2			557	
2	sasa222	302	1			557	
3	asdadkkakaka	302	2			557	
4	admin1234	302	3			557	
5	septe	302	1			557	
6	admin1234	302	1			557	
7	admin123	302	2			613	

Request

Pretty | Raw | Hex

```
POST /login HTTP/1.1
Host: 127.0.0.1:5000
Content-Length: 40
Cache-Control: max-age=0
sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Linux"
Accept-Language: en-US,en;q=0.9
Origin: http://127.0.0.1:5000
Content-Type: application/x-www-form-urlencoded
```

Response

Pretty | Raw | Hex | Render

```
HTTP/1.1 302 FOUND
Server: Werkzeug/3.1.8 Python/3.11.2
Date: Mon, 27 Apr 2026 20:54:13 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 207
Location: /dashboard
Vary: Cookie
Set-Cookie: session=
eyJfZmxhc2hlcyI6W3siIHQiOlsibWVzc2FnZSIsIkxvZ2dlZCBpbiBzdWNjZ
XNzZnVsbHkhIl19XSwicm9sZSI6Ik1RTkFHRVIiLCJ1c2VyX2lkIjoxfQ.ae
```

Inspector

Request attributes 2

Request body parameters 2

Request headers 19

Response headers 8

Payloads | Resource pool | Settings

4.5. Gestionarea Nesigura a Sesiunilor (OWASP: A7:2021)

Dupa ce un utilizator se autentifica cu succes, aplicatia trebuie sa mentina o "sesiune" pentru a-l recunoaste la cererile ulterioare, fara a-i cere parola la fiecare click. Flask gestioneaza acest lucru folosind un cookie de sesiune. Insa, modul in care sesiunile sunt configurate in v1 este nesigur si expune utilizatorii la mai multe riscuri.

Un atacator care obtine acces la codul sursa poate folosi aceasta cheie pentru a genera propriile cookie-uri de sesiune, impersonand orice utilizator.

Cookie-ul de sesiune (session) setat de aplicatie nu are flag-uri esentiale de securitate:

- **Secure**: Acest flag lipseste, ceea ce inseamna ca browser-ul va trimite cookie-ul si peste conexiuni necriptate (HTTP). Un atacator aflat in aceeasi retea (Man-in-the-Middle) poate intercepta cookie-ul si fura sesiunea.
- **HttpOnly**: Acest flag lipseste. Un cookie care nu este HttpOnly poate fi accesat de scripturi JavaScript care ruleaza pe pagina. Daca aplicatia ar avea o vulnerabilitate de Cross-Site Scripting (XSS), un atacator ar putea injecta un script care citeste document.cookie si trimite cookie-ul de sesiune catre serverul sau.

Sesiunile nu au o durata de viata definita si pot ramane valide pentru o perioada foarte lunga (sesiuni de tip "browser-session"). Daca un utilizator uita sa se delogheze de la un computer public, sesiunea sa ramane activa si poate fi preluata de urmatoarea persoana care foloseste browser-ul.

Poc: Ne logam in aplicatie cu un cont valid, de exemplu cosmin@test.com. Dupa login, deschidem uneltele de dezvoltator din browser (F12), mergem la tab-ul "Application" (in Chrome) sau "Storage" (in Firefox) si selectam "Cookies". Alternativ, putem intercepta raspunsul la cererea de login in Burp Suite.

Vom analiza header-ul Set-Cookie din raspunsul serverului sau detaliile cookie-ului stocat in browser.

Request

Pretty Raw Hex

```
1  POST /login HTTP/1.1
2  Host: 127.0.0.1:5000
3  Content-Length: 40
4  Cache-Control: max-age=0
5  sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
6  sec-ch-ua-mobile: ?0
7  sec-ch-ua-platform: "Linux"
8  Accept-Language: en-US,en;q=0.9
9  Origin: http://127.0.0.1:5000
10 Content-Type: application/x-www-form-urlencoded
11 Upgrade-Insecure-Requests: 1
12 User-Agent: Mozilla/5.0 (X11; Linux x86_64)
   AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0
   Safari/537.36
13 Accept:
   text/html,application/xhtml+xml,application/xml;q=0.9,image/a
   vif,image/webp,image/apng,*/*;q=0.8,application/signed-exchan
   ge;v=b3;q=0.7
14 Sec-Fetch-Site: same-origin
15 Sec-Fetch-Mode: navigate
16 Sec-Fetch-User: ?1
17 Sec-Fetch-Dest: document
18 Referer: http://127.0.0.1:5000/login
19 Accept-Encoding: gzip, deflate, br
20 Connection: keep-alive
21
22 email=admin%40test.com&password=admin123
```

Response

Pretty Raw Hex Render

```
1  HTTP/1.1 302 FOUND
2  Server: Werkzeug/3.1.8 Python/3.11.2
3  Date: Mon, 27 Apr 2026 21:20:24 GMT
4  Content-Type: text/html; charset=utf-8
5  Content-Length: 207
6  Location: /dashboard
7  Vary: Cookie
8  Set-Cookie: session=
   eyJfZmxhc2hlcyI6W3siIHQiOlsibWVzc2FnZSIsIkxvZ2dlZCBpbiBzdWNjZ
   XNzZnVsbHkhIl19XSwicm9sZSI6Ik1BTkFHRVIiLCJ1c2VyX2lkIjoxfQ.ae_
   TGA.2C69OgjaT_UYeXKonJBLmzBzOcM; HttpOnly; Path=/
9  Connection: close
10
11 <!doctype html>
12 <html lang=en>
13   <title>
       Redirecting...
     </title>
14   <h1>
       Redirecting...
     </h1>
15   <p>
       You should be redirected automatically to the target URL:
        <a href="/dashboard">
         /dashboard
       </a>
       . If not, click the link.
16
```

4.6. Resetarea Parolei Nesigura (OWASP: A7:2021)

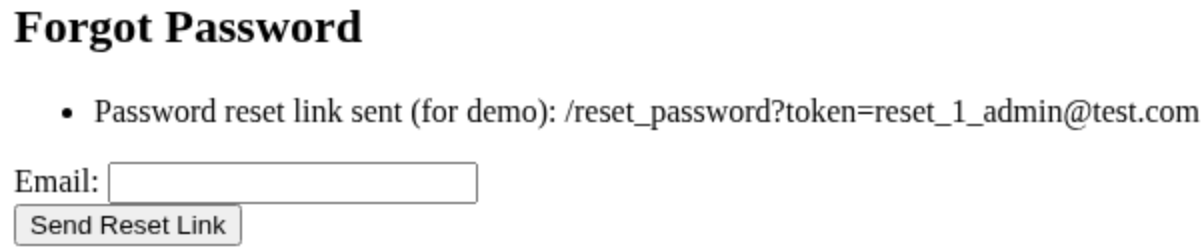
Functionalitatea de resetare a parolei este critica pentru experienta utilizatorului, dar in v1 este implementata intr-un mod extrem de periculos. Procesul se bazeaza pe generarea unor "token-uri" de resetare care sunt slabe, predictibile si nu au o durata de viata limitata.

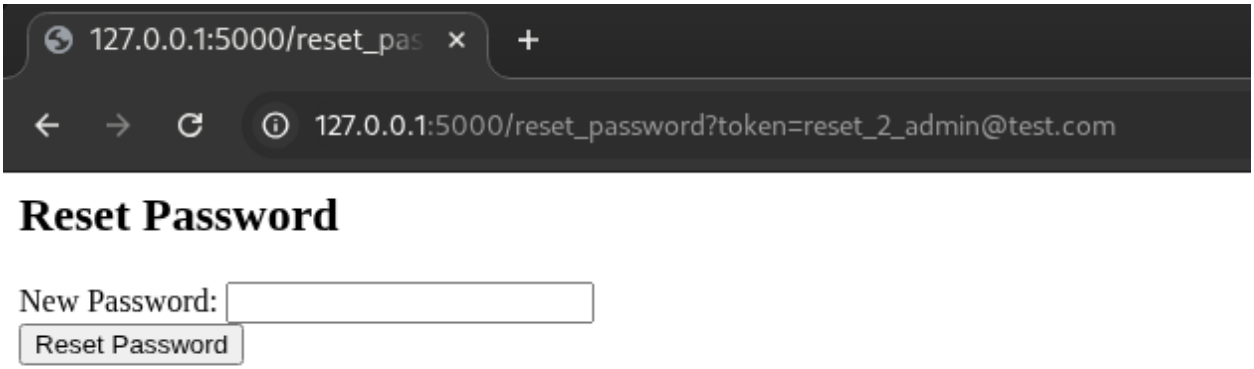
Token Predictibil: Structura token-ului este reset_{ID-ul utilizatorului}_{email-ul}. Un atacator care cunoaste (sau poate ghici) ID-ul si email-ul unui utilizator poate construi manual un token valid. ID-urile sunt adesea numere secventiale (1, 2, 3...), iar email-urile pot fi descoperite prin User Enumeration.

Lipsa Expirarii: Token-ul generat nu are o data de expirare. Odata creat, ramane valid pe termen nelimitat. Daca un astfel de token este expus (de exemplu, intr-un log, intr-un email interceptat sau in istoricul browser-ului), el poate fi folosit oricand in viitor pentru a prelua controlul contului.

Lipsa Semnaturii Criptografice: Token-ul nu este semnat digital, ceea ce inseamna ca aplicatia nu are nicio metoda de a verifica daca token-ul a fost generat de ea sau a fost fabricat de un atacator.

Demonstratia consta in fabricarea manuala a unui token de resetare pentru un cont tinta, de exemplu, contul de administrator (admin@test.com), presupunand ca stim ca are id=1.





5. Implementarea Masurilor de Securitate - v2

5.1. Password Policy

Pentru a adresa vulnerabilitatile critice de Weak Password Policy si Plain Text Password Storage, am implementat o solutie pe doua niveluri in fisierul main.py:

- Am impus o politica de complexitate a parolelor la inregistrare.
- Am folosit biblioteca Flask-Bcrypt pentru a stoca parolele sub forma de hash-uri securizate.

Desi nu este strict necesar pentru bcrypt, este o buna practica sa marim lungimea campului password pentru a ne asigura ca poate stoca hash-uri de lungimi diferite, generate de diversi algoritmi.

In ruta /register, inainte de a crea utilizatorul, am adaugat o functie de validare a parolei. Aceasta functie verifica daca parola respecta un set de reguli de complexitate. Am folosit expresii regulate (regex) pentru o verificare eficienta.
In ruta /login, procesul de verificare a parolei este complet schimbat. Nu mai comparam doua string-uri in clar. In schimb:

- Cautam utilizatorul in baza de date dupa email.
- Daca utilizatorul exista, folosim functia bcrypt.check_password_hash() pentru a compara in mod sigur parola introdusa de utilizator cu hash-ul stocat in baza de date.

Aceste masuri elimina complet riscul expunerii parolelor in cazul unei brese a bazei de date si reduc semnificativ sansele unui atac de tip Brute-Force prin impunerea unor parole complexe.

POZA BD CU HASH DIN V2.DB

5.2. Prevenire User Enumeration

Vulnerabilitatea de User Enumeration era cauzata de mesajele de eroare diferite returnate de formularul de login. Remedierea consta in standardizarea acestor mesaje, astfel incat un atacator sa nu mai poata distinge intre un email invalid si o parola incorecta.

Am modificat logica din functia de login din main.py pentru a returna un singur mesaj de eroare generic in ambele situatii de esec: fie ca email-ul nu exista, fie ca parola este gresita.

Prin aceasta modificare, indiferent daca un atacator introduce un email valid cu parola gresita sau un email complet invalid, el va primi intotdeauna acelasi raspuns si va vedea acelasi mesaj: "Invalid email or password.".

PoC: Configuram Burp Intruder sa testeze aceeasi lista de email-uri (valide si invalide) pe endpoint-ul de login al aplicatiei v2. Dupa finalizarea atacului, analizam din nou rezultatele. De data aceasta, vom observa ca toate cererile esuate au un raspuns identic. Nu mai exista nicio diferenta in lungimea (Length) raspunsului HTTP sau in continutul acestuia.

Sniper attack | Start attack

Target: http://127.0.0.1:5001 | Update Host header to match target

Positions: Add § | Clear § | Auto §

```
1  POST /login HTTP/1.1
2  Host: 127.0.0.1:5001
3  Content-Length: 38
4  Cache-Control: max-age=0
5  sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
6  sec-ch-ua-mobile: ?0
7  sec-ch-ua-platform: "Linux"
8  Accept-Language: en-US,en;q=0.9
9  Origin: http://127.0.0.1:5001
10 Content-Type: application/x-www-form-urlencoded
11 Upgrade-Insecure-Requests: 1
12 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36
   (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
13 Accept:
   text/html,application/xhtml+xml,application/xml;q=0.9,image/avif
   ,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b
   3;q=0.7
14 Sec-Fetch-Site: same-origin
15 Sec-Fetch-Mode: navigate
16 Sec-Fetch-User: ?1
17 Sec-Fetch-Dest: document
18 Referer: http://127.0.0.1:5001/login
19 Accept-Encoding: gzip, deflate, br
20 Connection: keep-alive
21
22 email=§admin%40test.com§&password=asdads
```

Search | 1 highlight | 1 payload position | Length: 838

Payloads

Payload position: All payload positions

Payload type: Simple list

Payload count: 6

Request count: 6

Payload configuration

This payload type lets you configure a simple list of strings that are used as payloads.

Paste | Load... | Remove | Clear | Deduplicate | Add

mamsam@test.com
sadadasa
admin@test.com
cosmin@test.com
test@test.com

Add from list... [Pro version only]

Payload processing

You can define rules to perform various processing tasks on each payload before it is used.

Add | Edit | Remove | Up

Enabled	Rule

Payloads | Resource pool | Settings

Attack | Save | 6. Intruder attack of http://127.0.0.1:5001

6. Intruder attack of http://127.0.0.1:5001

Attack | Save

Results | Positions

Capture filter: Capturing all items | Apply capture filter

View filter: Showing all items

Request	Payload	Status code	Response received	Error	Timeout	Length	Comment
0		200	13			898	
1	test@test.com	200	7			898	
2	admin@test.com	200	5			898	
3	atasaA@test.com	200	3			898	
4	sadadasdaa@@D.COMA	200	2			898	
5	cosmin@test.com	200	3			898	

Payloads | Resource pool | Settings

5.3. Protectie Impotriva Atacurilor Brute-Force

Pentru a oferi o protectie robusta impotriva atacurilor de tip Brute-Force, am implementat o strategie de aparare in adancime (defense-in-depth) in v2, care combina trei mecanisme de control:

- Rate Limiting la nivel de IP: Limiteaza numarul de cereri pe care o singura adresa IP le poate face catre endpoint-ul de login.
- Blocarea Contului (Account Lockout): Blocheaza temporar un cont specific dupa un numar de incercari de login esuate.
- Blocarea IP-ului Atacator (IP Blacklisting): Un mecanism dinamic care blocheaza complet adresa IP a unui atacator care a reusit sa blocheze un cont, impiedicandu-l sa incerce sa atace alte conturi.

PoC: Vom relua atacul de Brute-Force cu Burp Intruder pe aplicatia v2 pentru a demonstra eficienta masurilor.

Testarea Rate Limiting:
Lansam atacul cu o viteza mare. Dupa primele 10 cereri intr-un minut, Burp Intruder va incepe sa primeasca raspunsuri cu codul de stare 429 Too Many Requests. Atacul este oprit eficient.

Testarea Account Lockout + Ip Lockoutt:

Configuram Burp Intruder sa ruleze cu o viteza mica (ex: o cerere la 10 secunde) pentru a nu declansa Rate Limiting-ul.Tinta este un cont valid. Dupa a 3-a incercare esuata (MAX_FAILED_ATTEMPTS), chiar daca o cerere ulterioara ar folosi parola corecta, login-ul va esua si se va afisa mesajul de cont blocat. In acelasi timp, adresa IP a atacatorului este adaugata in ip_blacklist. Orice cerere ulterioara de la acest IP (a 6-a, a 7-a etc.), indiferent daca incearca sa se logheze pe admin@test.com sau pe alt cont precum cosmin@test.com, va primi imediat un raspuns 429 Too Many Requests de la decoratorul check_ip_blacklist, chiar inainte ca logica de login sa fie executata.

Burp Project Intruder Repeater View Help — Burp Suite Community Edition v2026.3.3 - Temporary Project

Dashboard | Target | Proxy | Intruder | Repeater | Collaborator | Sequencer | Decoder | Comparer | Logger | Organizer | Extensions | Discover

1 | 2 | 3

Sniper attack — Start attack

Target: http://127.0.0.1:5001 — Update Host header to match target

Positions: Add § | Clear § | Auto §

```
POST /login HTTP/1.1
Host: 127.0.0.1:5001
Content-Length: 48
Cache-Control: max-age=0
sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Linux"
Accept-Language: en-US,en;q=0.9
Origin: http://127.0.0.1:5001
Content-Type: application/x-www-form-urlencoded
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif
,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b
3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer: http://127.0.0.1:5001/login
Accept-Encoding: gzip, deflate, br
Connection: keep-alive

email=admin%40test.com&password=§admin%40test.com§
```

Search — 1 highlight | 1 payload position | Length: 848

Payloads

Payload position: All payload positions
Payload type: Simple list
Payload count: 6
Request count: 6

Payload configuration

This payload type lets you configure a simple list of strings that are used as payloads.

Paste | Load... | Remove | Clear | Deduplicate | Add

test@test.com
admin@test.com
atasaA@test.com
sadadasdaa@@D.COMA
cosmin@test.com
AdminPass123!

Enter a new item

Add from list... [Pro version only]

Payload processing

You can define rules to perform various processing tasks on each payload before it is used.

Add | Edit | Remove | Enabled | Rule

Attack Save — 8. Intruder attack of http://127.0.0.1:5001

8. Intruder attack of http://127.0.0.1:5001 — Attack | Save

Results | Positions

Capture filter: Capturing all items — Apply capture filter

View filter: Showing all items

Request	Payload	Status code	Response received	Error	Timeout	Length	Comment
0		302	289			678	
1	test@test.com	302	241			678	
2	admin@test.com	200	4			898	
3	atasaA@test.com	200	3			898	
4	sadadasdaa@@D.COMA	200	3			898	
5	cosmin@test.com	200	3			898	
6	AdminPass123!	200	3			898	

Request — Pretty | Raw | Hex

```
POST /login HTTP/1.1
Host: 127.0.0.1:5001
Content-Length: 49
Cache-Control: max-age=0
sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Linux"
Accept-Language: en-US,en;q=0.9
Origin: http://127.0.0.1:5001
Content-Type: application/x-www-form-urlencoded
```

Response — Pretty | Raw | Hex | Render

```
</h2>

<div class="error">
  Your IP is temporarily blocked due to too many failed login
  attempts.
</div>
```

Inspector

Request attributes 2
Request body parameters 2
Request headers 19
Response headers 10

```
cosmin-caravan@lab-sec:~/Downloads/auth-security-lab/v2$ sqlite3 v2.db
SQLite version 3.40.1 2022-12-28 14:03:47
Enter ".help" for usage hints.
sqlite> select * from users;
1|admin@test.com|$2b$12$cXt7/O1k6ubkSevDw3xi7OJ9WQ3PMYaGoAz5SPtkUMBprC9l.uwe2|MANAGER|2026-04-28 06:41:42|3|2026-04-28 06:54:54.599383
2|cosmin@test.com|$2b$12$faKluKV0T7U9uJeFdKS6eu9AAJ42MX2gkhmrom93Y.WmHldPCxoS.|ANALYST|2026-04-28 06:41:42|0|
sqlite>
```

1 Hidden T...

5.4. Gestionarea Securizata a Sesiunilor

In versiunea v1, cookie-urile de sesiune erau vulnerabile din cauza unei chei secrete slabe si a lipsei flag-urilor de securitate. In v2, am implementat o serie de masuri pentru a intari securitatea sesiunilor, conform bunelor practici. Am eliminat cheia secreta hardcodata din fisierul main.py. In schimb, aplicatia este configurata sa citeasca cheia dintr-o variabila de mediu. Acest lucru respecta principiul de a separa configuratia de cod.

Am configurat aplicatia Flask sa adauge automat flag-urile esentiale de securitate pentru cookie-ul de sesiune.

- SESSION_COOKIE_SECURE = True: Acest flag forteaza browser-ul sa trimita cookie-ul doar prin conexiuni securizate (HTTPS). Previne interceptarea cookie-ului intr-un atac Man-in-the-Middle pe o retea Wi-Fi publica, de exemplu.
- SESSION_COOKIE_HTTPONLY = True: Acest flag interzice accesul la cookie prin JavaScript (document.cookie). Este o masura cruciala de aparare impotriva atacurilor de tip Cross-Site Scripting (XSS) care au ca scop furtul sesiunii.
- SESSION_COOKIE_SAMESITE = 'Lax': Acest flag ofera o protectie buna impotriva atacurilor de tip Cross-Site Request Forgery (CSRF), asigurand ca cookie-ul este trimis doar atunci cand utilizatorul navigheaza direct catre site-ul nostru, nu si cand este initiata o cerere de pe un alt site.

Am configurat sesiunile sa expire automat dupa o perioada de inactivitate (de exemplu, 30 de minute), fortand utilizatorul sa se re-autentifice.

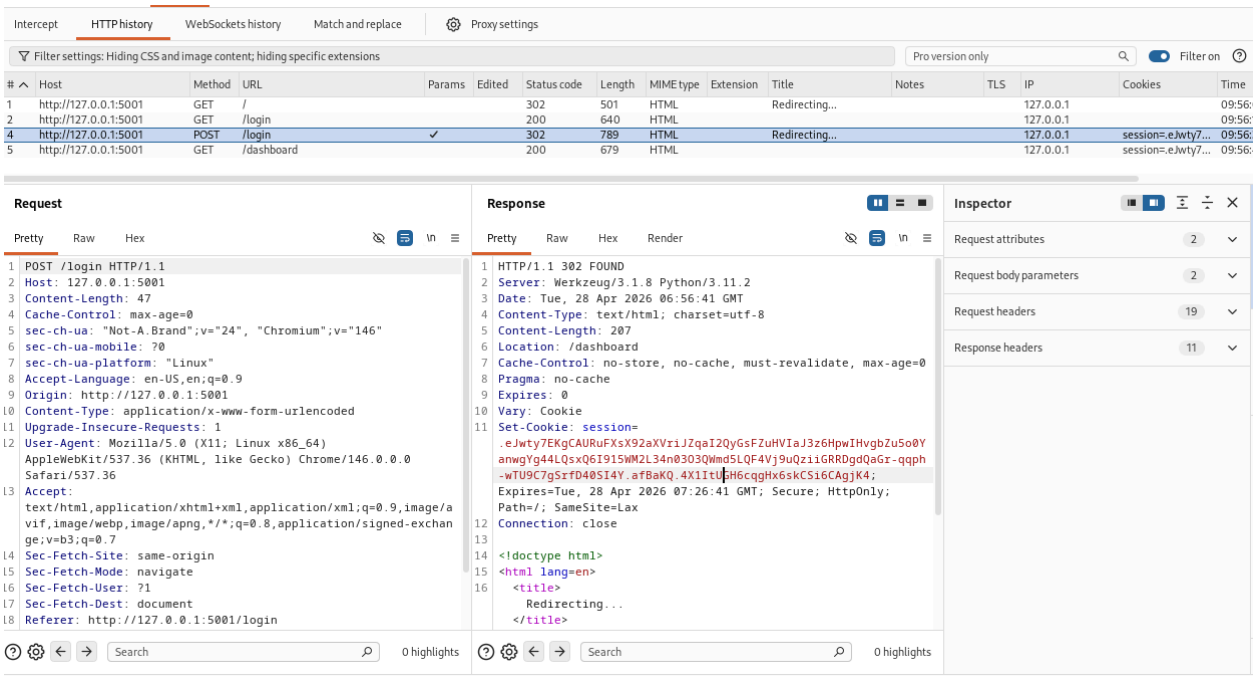
PoC:

Pentru a demonstra ca masurile de securitate functioneaza, vom inspecta din nou cookie-ul de sesiune dupa un login reusit in aplicatia v2.

Folosind uneltele de dezvoltator din browser sau interceptand raspunsul in Burp Suite, analizam header-ul Set-Cookie.

Analiza observatiilor:

- Expires/Max-Age: Cookie-ul are acum o data de expirare clara, setata la 30 de minute
- Secure: Flag-ul este acum bifat (true).
- HttpOnly: Flag-ul este acum bifat (true).
- SameSite: Valoarea este setata la Lax.



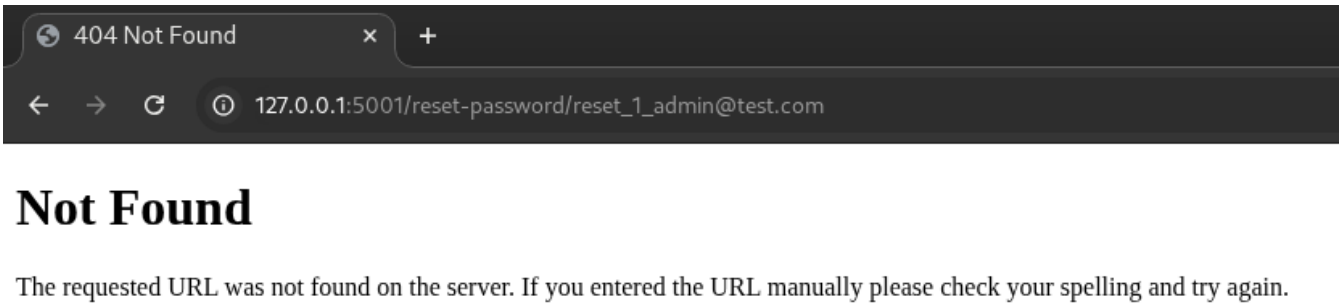
5.5. Mecanism Sigur de Resetarea Parolei

Vulnerabilitatea din v1 provenea de la utilizarea unor token-uri de resetare predictibile, fara semnatura si fara data de expirare. Pentru a remedia acest lucru, am inlocuit complet mecanismul vechi cu unul bazat pe biblioteca itsdangerous, care este special conceputa pentru a genera si verifica token-uri securizate in aplicatii web. Biblioteca itsdangerous este deja o dependinta a Flask, deci nu necesita o instalare separata. Solutia se bazeaza pe crearea unui "serializer" care poate transforma date (cum ar fi ID-ul unui utilizator) intr-un token semnat criptografic si cu timp de expirare. Biblioteca itsdangerous este deja o dependinta a Flask, deci nu necesita o instalare separata. Solutia se bazeaza pe crearea unui "serializer" care poate transforma date (cum ar fi ID-ul unui utilizator) intr-un token semnat criptografic si cu timp de expirare.

PoC: Incercam sa construim manual un URL de resetare, cum am facut in v1: http://127.0.0.1:5000/reset-password/reset_1_admin@test.com.
Aplicatia va intra in blocul except deoarece token-ul nu are o semnatura valida si va afisa mesajul "The password reset link is invalid.". Atacul esueaza.

Generam un token valid.

Asteptam sa treaca perioada de valabilitate (30 de minute in exemplul nostru).



Incercam sa accesam URL-ul cu token-ul expirat. Aplicatia va prinde eroarea SignatureExpired si va afisa mesajul "The password reset link has expired.". Atacul esueaza.

```
(venv) cosmin-caravan@lab-sec:~/Downloads/auth-security-lab/v2$ python3 main.py
127.0.0.1 - - [28/Apr/2026 10:05:20] "GET / HTTP/1.1" 302 -
127.0.0.1 - - [28/Apr/2026 10:05:20] "GET /dashboard HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 10:05:22] "GET /logout HTTP/1.1" 302 -
127.0.0.1 - - [28/Apr/2026 10:05:22] "GET /login HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 10:05:23] "GET /forgot_password HTTP/1.1" 200 -
Reset URL (do not show in production): http://127.0.0.1:5001/reset_password/ImFkbWluQHRlc3QuY29tIg.afBcOQ.b1lvPElnZkt9DJHFjX2H
127.0.0.1 - - [28/Apr/2026 10:05:29] "POST /forgot_password HTTP/1.1" 302 -
127.0.0.1 - - [28/Apr/2026 10:05:29] "GET /forgot_password HTTP/1.1" 200 -
```

Prin utilizarea itsdangerous, am eliminat complet posibilitatea de a fabrica sau refolosi token-uri de resetare, asigurand ca doar utilizatorul legitim, in posesia unui link valid si recent, isi poate reseta parola.

6. Audit si Logging

Un aspect fundamental al securitatii, adesea neglijat, este capacitatea de a inregistra si analiza evenimentele critice care au loc in aplicatie. Un sistem de audit si logging robust (jurnalizare) este esential pentru:

- Detectia Incidentelor: Identificarea in timp real sau post-factum a unor activitati suspecte sau a unor atacuri in desfasurare.
- Analiza Post-Incident (Forensics): Intelegerea modului in care a avut loc un atac, ce date au fost compromise si ce actiuni a intreprins atacatorul.
- Trasabilitate (Accountability): Atribuirea actiunilor unor utilizatori specifici.

- Conformitate (Compliance): Multe standarde de securitate (precum PCI-DSS sau GDPR) impun existenta unor jurnale de audit detaliate.

In v1, desi tabela audit_logs exista in schema bazei de date, ea era subutilizata, iar evenimentele critice de securitate nu erau inregistrate. In v2, am implementat o logica de logging functionala.

PoC:
Pentru a demonstra eficienta mecanismului de audit, simulam un scenariu de atac esuat si apoi analizam jurnalul. Un atacator cu adresa IP 127.0.0.1 incearca sa ghiceasca parola contului admin@test.com. Dupa 5 incercari esuate, contul este blocat.

Dupa simularea acestui scenariu, interogam tabela audit_logs pentru a vedea ce s-a inregistrat.

```
cosmin-caravan@lab-sec:~/Downloads/auth-security-lab/v2$ sqlite3 v2.db
SQLite version 3.40.1 2022-12-28 14:03:47
Enter ".help" for usage hints.
sqlite> select * from audit_logs;
1|1|LOGIN_SUCCESS|||2026-04-28 06:56:41|127.0.0.1
2|1|LOGOUT|||2026-04-28 07:05:22|127.0.0.1
3|1|LOGIN_FAILURE|||2026-04-28 07:07:57|127.0.0.1
4|1|LOGIN_FAILURE|||2026-04-28 07:08:00|127.0.0.1
5|1|ACCOUNT_LOCKED|||2026-04-28 07:08:02|127.0.0.1
6|1|LOGIN_FAILURE|||2026-04-28 07:08:02|127.0.0.1
sqlite>
```

Aceasta vizibilitate este cruciala. Un administrator de sistem care monitorizeaza aceste log-uri ar putea configura alerte pentru a fi notificat la un numar mare de LOGIN_FAILURE intr-un interval scurt, putand astfel sa reactioneze rapid la un atac in desfasurare. In cazul unui incident reusit, acest jurnal ar fi punctul de plecare pentru a intelege cum a patruns atacatorul in sistem.

7. Concluzii

Acest proiect mi-a oferit o perspectiva practica asupra mentalitatii unui atacator si, mai important, asupra responsabilitatii unui dezvoltator de a construi aplicatii sigure si de incredere. Am inteles ca securitatea nu este doar despre a scrie cod care functioneaza, ci despre a anticipa modurile in care acesta ar putea fi abuzat si a construi preventiv aparari robuste.